Clase 3
Desarrollo moderno de software

Objetivos de la Clase

- Explorar *DevOps* como cultura para romper silos y acelerar valor
- Intro a *Team Topologies* como herramienta para entender la colaboración entre equipos
- Entender *Continuous Integration* para feedback rápido y calidad continua.
- Entender *Continuous Delivery* para despliegues seguros y sin fricción.

Agenda

1. DevOps
2. Continuous Integration
3. Continuous Delivery

Agenda

1. **DevOps**
2. Continuous Integration
3. Continuous Delivery

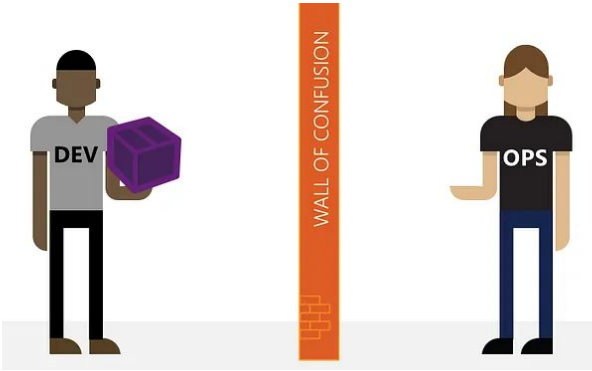
Environments

- Environments in SDLC
  - Development → Local coding & debugging → Sometimes it doesn't work occasionally
  - Testing/QA → Runs automated/manual tests →Should work most of the time
  - Staging → Production-like, for final checks, sometimes pentesting or stress testing → Should work always
  - Production → Live system for users → MUST WORK

Motivations

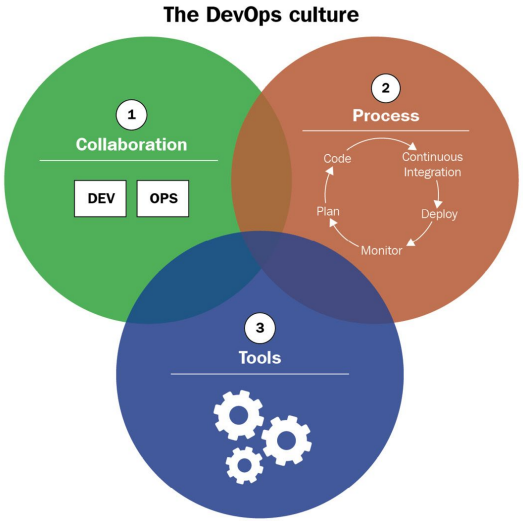
- Difficulties of traditional sw development:
  - Multiple manual processes requiring specific tinkering
  - Lack of collaboration and communication between teams
- Effects:
  - Slow software delivery and Delivery cycles
  - Frequent errors and instability





DevOps to the Rescue

- Definition
  - A **culture and mindset** that bridges development (Dev) and operations (Ops)
  - It emphasizes **shared responsibility** and **collaboration**.
- Key Goals
  - **Faster** time to market
  - Higher **reliability** & **quality**
  - Better **alignment** between business & tech
  - **Lower operational risk** & better throughput



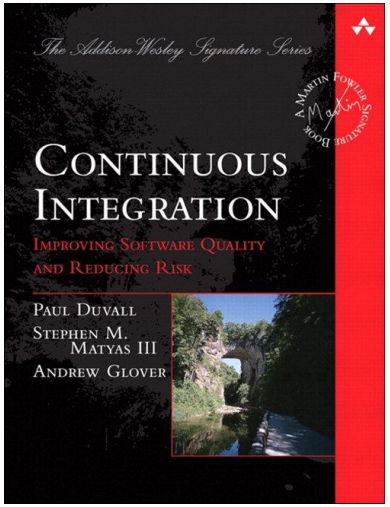


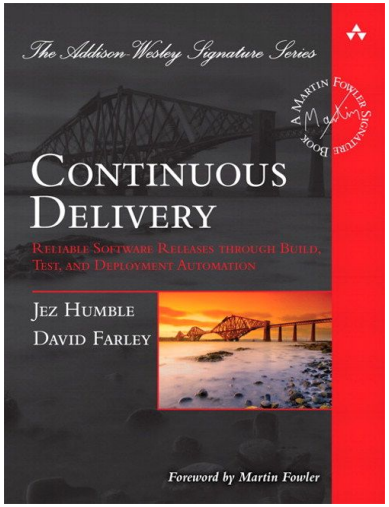
Principles

- **Shared Responsibility**
  - Dev and Ops teams jointly own the product from build through operation.
- **Fast Feedback Loops**
  - Use frequent feedback & iteration to improve continuously.
- **Automation**
  - Automate builds, tests, Deliverys, and infrastructure as much as possible.
- **Continuous Integration / Delivery (CI/CD)**
  - Integrate and deliver changes in small batches frequently.
- **Culture & Trust**
  - Embrace failure, encourage learning, and support open communication.

Principles (alt)

- Build **quality** in
- Work in **small batches**
- **Computers do repetitive** tasks, **people solve problems**
- Relentlessly pursue **continuous improvement**
- **Everyone is responsible**





Patterns

Dev & Ops Collaboration

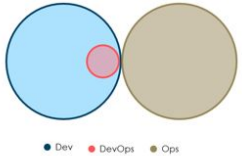
As-Is Model



- Need substantial organization change to establish it and a good degree of technical competency in each product team
- Difficult to scale at enterprise level
- Requires high level of collaboration
- Chances of failures are high if shared goals are not clearly owned by development and operations team
- Team needs to acquire required skills to understand each other's work and seek inputs
- Difficult to achieve standardization of process & tooling, as team do not have an organization level visibility

Embedded Infra DevOps

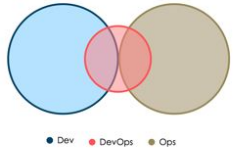
Potential Target



- Platform team delivers an agile DevOps infrastructure (IaaS) for multiple product teams to consume
- Team within dev owns and manages operational features, provisioning, metrics, monitoring
- This team communicates with Platform Ops (IaaS) team
- Less collaboration, easy to implement, quick value
- Requires initial maturity to have a stable platform with a dedicate Platform Ops team
- Difficult to scale and standardize across enterprise if initial maturity is low

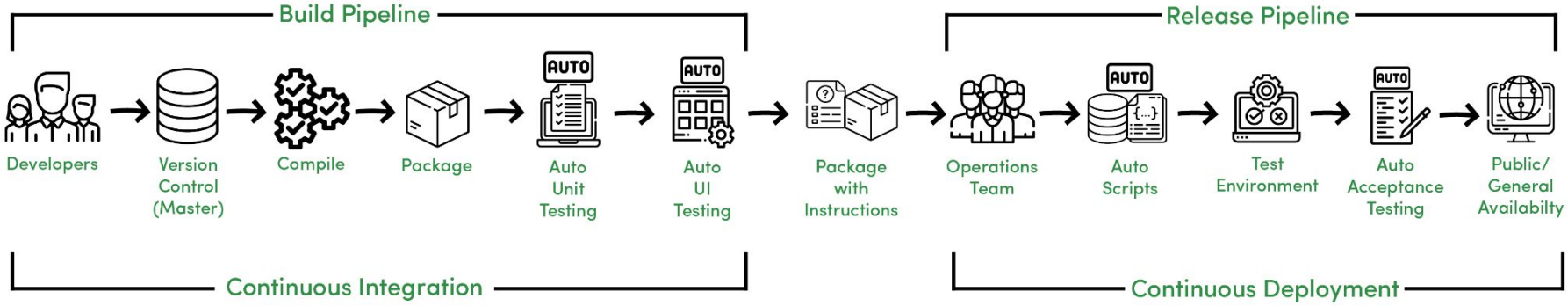
DevOps As Shared Service

Potential Target

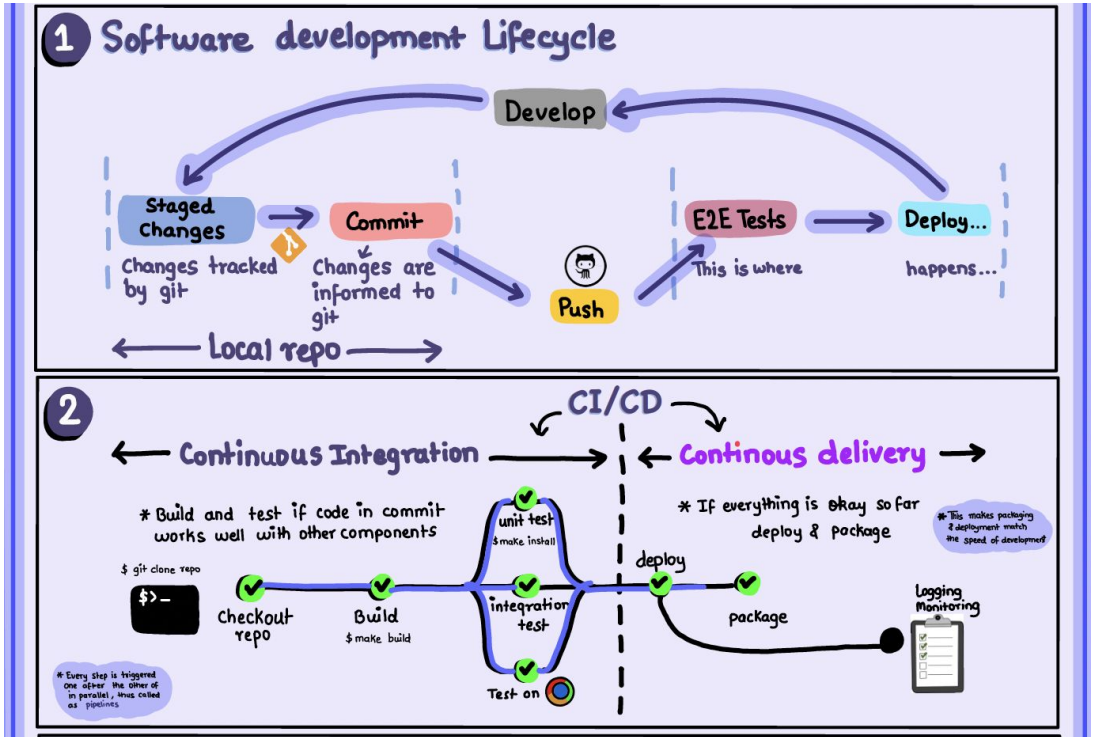


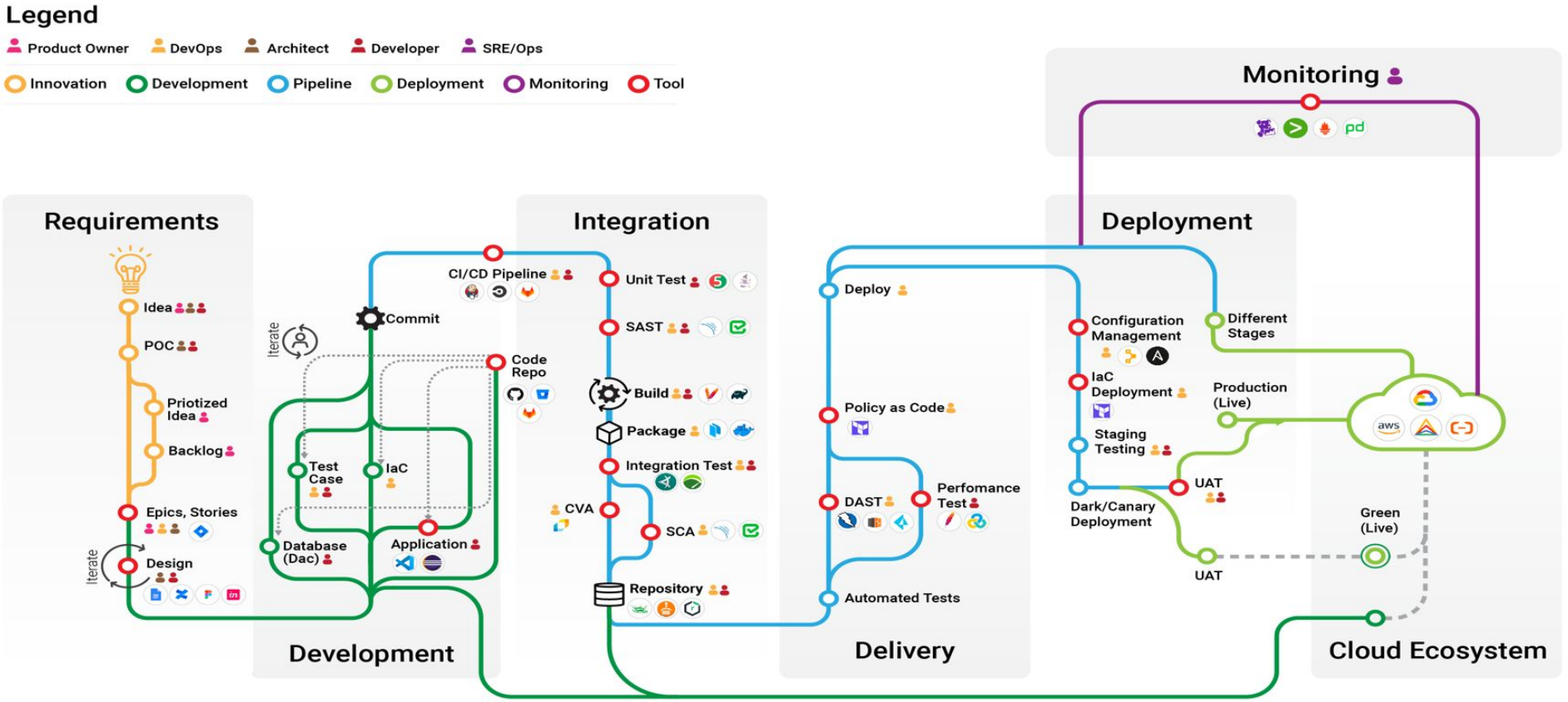
- Suitable for organizations that tend to have(or tendency) gap between Development & Operations team
- Well-defined ownership to drive agility and DevOps practices and adoption across the organization
- Helps standardize the process, tool chain
- Focus on automation-first DevSecOps framework
- Brings strategy and execution under one umbrella to improve visibility across SDLC
- A great way to manage scale and stability
- Product team not fully autonomous

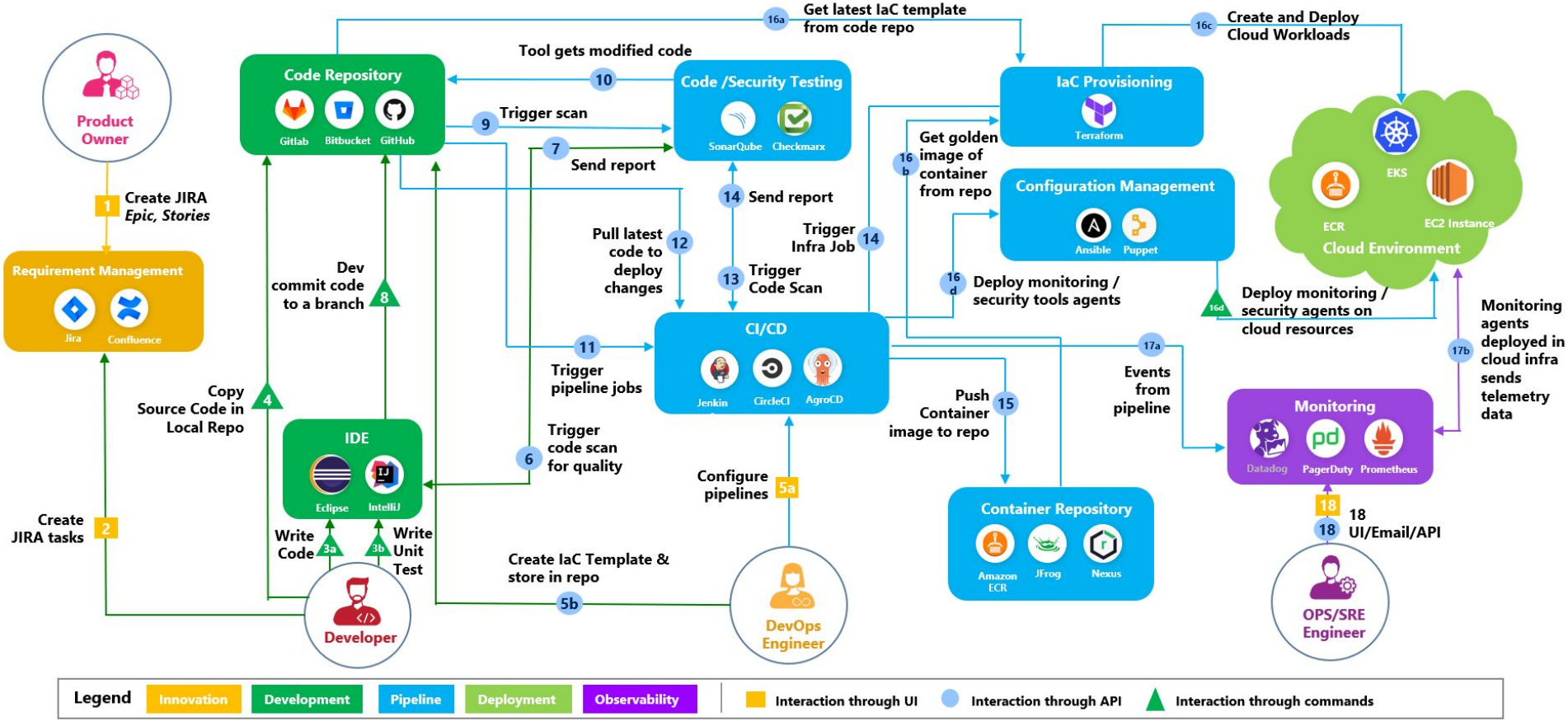
CI/CD


Build Pipeline
Release Pipeline
Developers
Version Control (Master)
Compile
Package
AUTO
Auto Unit Testing
AUTO
Auto UI Testing
Package with Instructions
Operations Team
AUTO
Auto Scripts
Test Environment
AUTO
Auto Acceptance Testing
Public/ General Availabilty
Continuous Integration
Continuous Deployment

CI/CD


1
Software development Lifecycle
Develop
Staged Changes
Changes tracked by git
Commit
Changes are informed to git
Push
E2E Tests
This is where
Deploy...
happens...
Local repo
2
CI/CD
Continuous Integration
Continous delivery
* Build and test if code in commit works well with other components
$ git clone repo
$>_
Checkout repo
Build
$ make build
unit test
$ make install
integration test
Test on
* If everything is okay so far deploy & package
deploy
package
Logging
Monitoring
* This makes packaging & deployment match the speed of development
* Every step is triggered one after the other or in parallel, thus called as pipelines

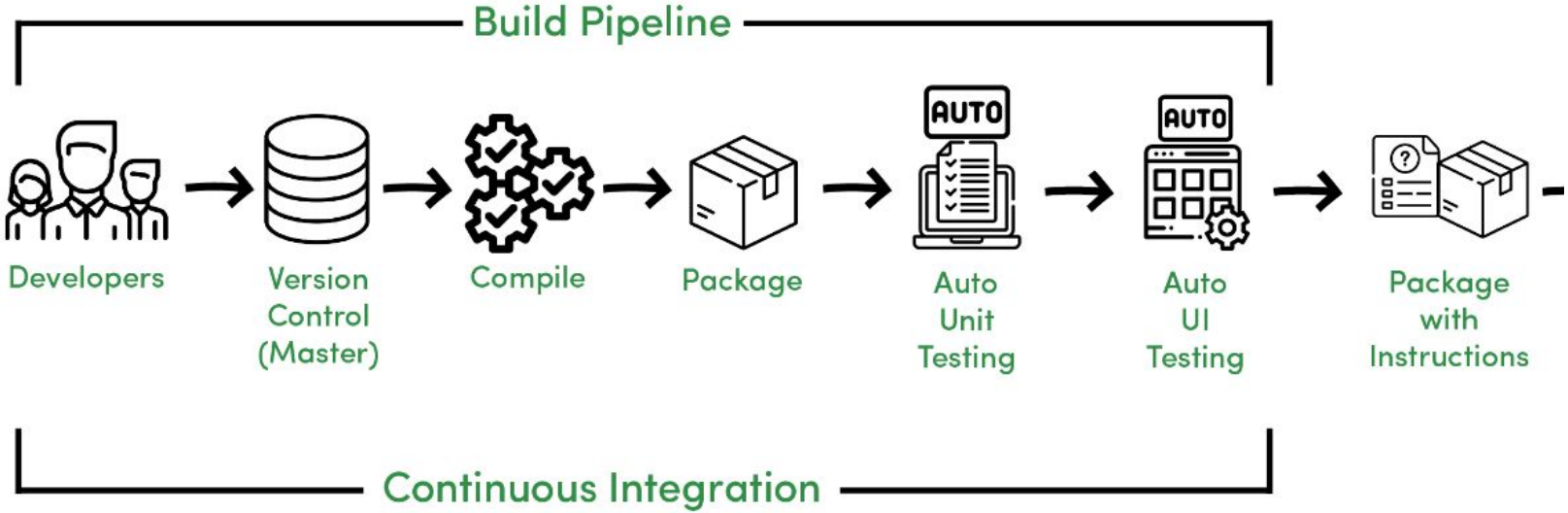

Legend
Product Owner
DevOps
Architect
Developer
SRE/Ops
Innovation
Development
Pipeline
Deployment
Monitoring
Tool
Monitoring
Requirements
Idea
POC
Priotized Idea
Backlog
Epics, Stories
Design
Iterate
Development
Commit
Iterate
Code Repo
Test Case
IaC
Database (Dac)
Application
Integration
CI/CD Pipeline
Unit Test
SAST
Build
Package
Integration Test
CVA
SCA
Repository
Deploy
Policy as Code
DAST
Perfomance Test
Automated Tests
Delivery
Deployment
Configuration Management
IaC Deployment
Staging Testing
Dark/Canary Deployment
Different Stages
Production (Live)
UAT
Green (Live)
UAT
Cloud Ecosystem


Product Owner
1 Create JIRA Epic, Stories
Requirement Management
Jira
Confluence
2 Create JIRA tasks
Code Repository
Gitlab
Bitbucket
GitHub
16a Get latest IaC template from code repo
Tool gets modified code
10
Code /Security Testing
SonarQube
Checkmarx
9 Trigger scan
7 Send report
14 Send report
Pull latest code to deploy changes 12
Trigger Infra Job 14
13 Trigger Code Scan
Dev commit code to a branch 8
4 Copy Source Code in Local Repo
11 Trigger pipeline jobs
CI/CD
Jenkin
CircleCI
AgroCD
6 Trigger code scan for quality
IDE
Eclipse
IntelliJ
Write Code 3a
Write Unit Test 3b
Configure pipelines 5a
Create IaC Template & store in repo 5b
Developer
DevOps Engineer
IaC Provisioning
Terraform
16c Create and Deploy Cloud Workloads
16 b Get golden image of container from repo
Configuration Management
Ansible
Puppet
16 d Deploy monitoring / security tools agents
16d Deploy monitoring / security agents on cloud resources
EKS
ECR
EC2 Instance
Cloud Environment
17b Monitoring agents deployed in cloud infra sends telemetry data
17a Events from pipeline
Push Container image to repo 15
Container Repository
Amazon ECR
JFrog
Nexus
Monitoring
Datadog
PagerDuty
Prometheus
18 18 UI/Email/API
OPS/SRE Engineer
Legend
Innovation
Development
Pipeline
Deployment
Observability
Interaction through UI
Interaction through API
Interaction through commands

Agenda

1. DevOps
2. **Continuous Integration**
3. Continuous Delivery

Continuous Integration



Continuous Integration

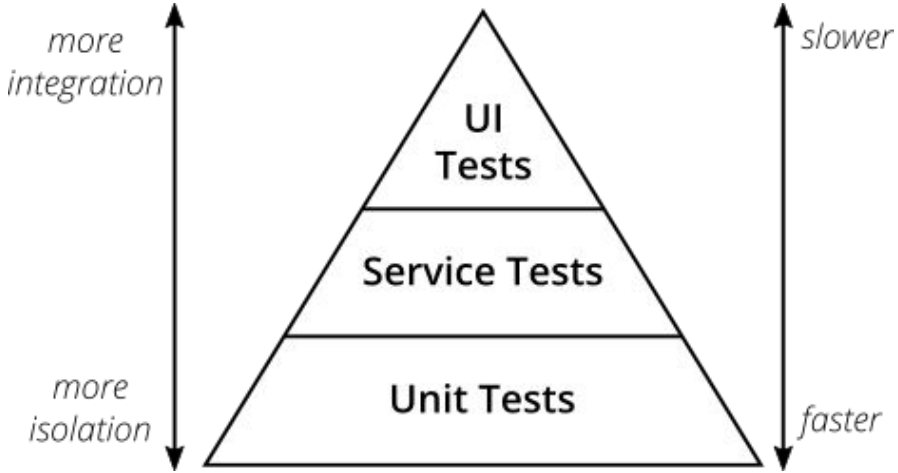
- What is it?
  - **CI is a practice where developers frequently merge their code changes into a central repository** where **automated build and test are run**.
- Motivation
  - **Reduced risk of delivery delays**
  - **Less time** wasted in integration
  - **Less bugs** and higher software quality
  - **Enables Refactoring** for sustained productivity
  - **Release to Production is a business decision**
- The core idea
  - *Integrate early and often to avoid integration nightmares*.

CI: Practices

- Build and test on every change
- Test result on every change
- Fix test failures quickly
- Keep in mind the test pyramid (more on this later)

Test Pyramid

Is a way of thinking about **how different kinds of automated tests should be used** to create a balanced portfolio.



Not CI but Good Friends: Introducing linters and such

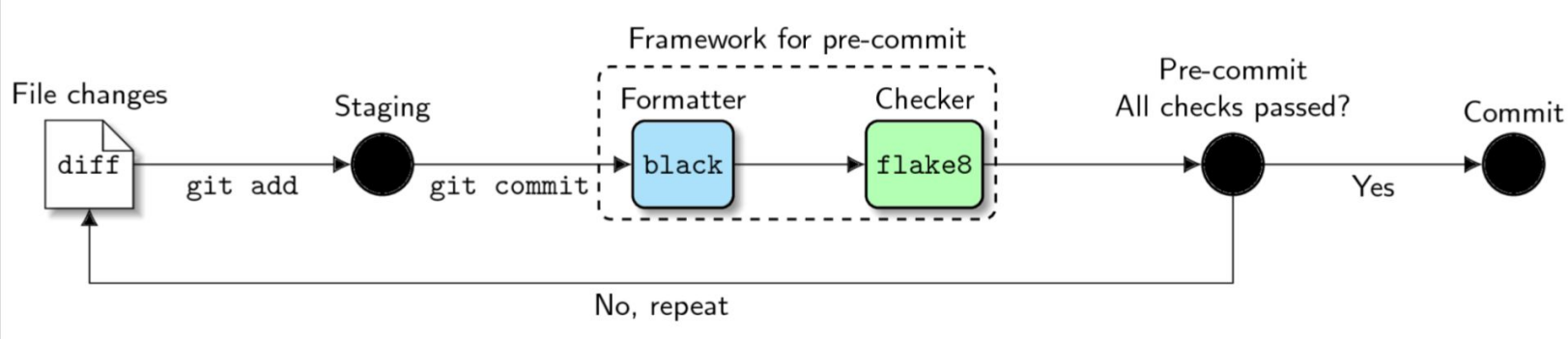
They are tools that can be used to improve code quality locally and can be integrated into the CI as a form of validation.

Some examples:

- Linter: Detects errors, enforces coding standards (pylint)
- Formatters: Formats code consistently (black)
- Security verification: Detects security vulnerabilities (bandit)
- Import order (or being just pedantic): Organizes imports alphabetically order (isort)

Not CI but Good Friends: Pre Commit hooks

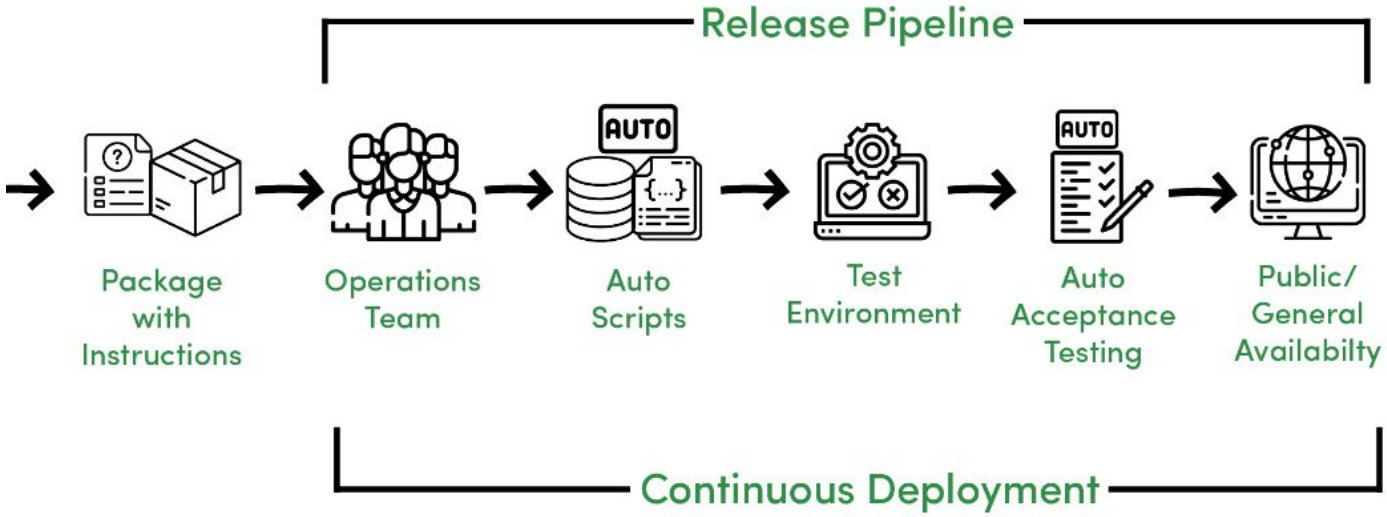
A nice git functionality that allows to run actions on each commit locally.



Agenda

1. DevOps
2. Continuous Integration
3. **Continuous Delivery**

Continuous Delivery



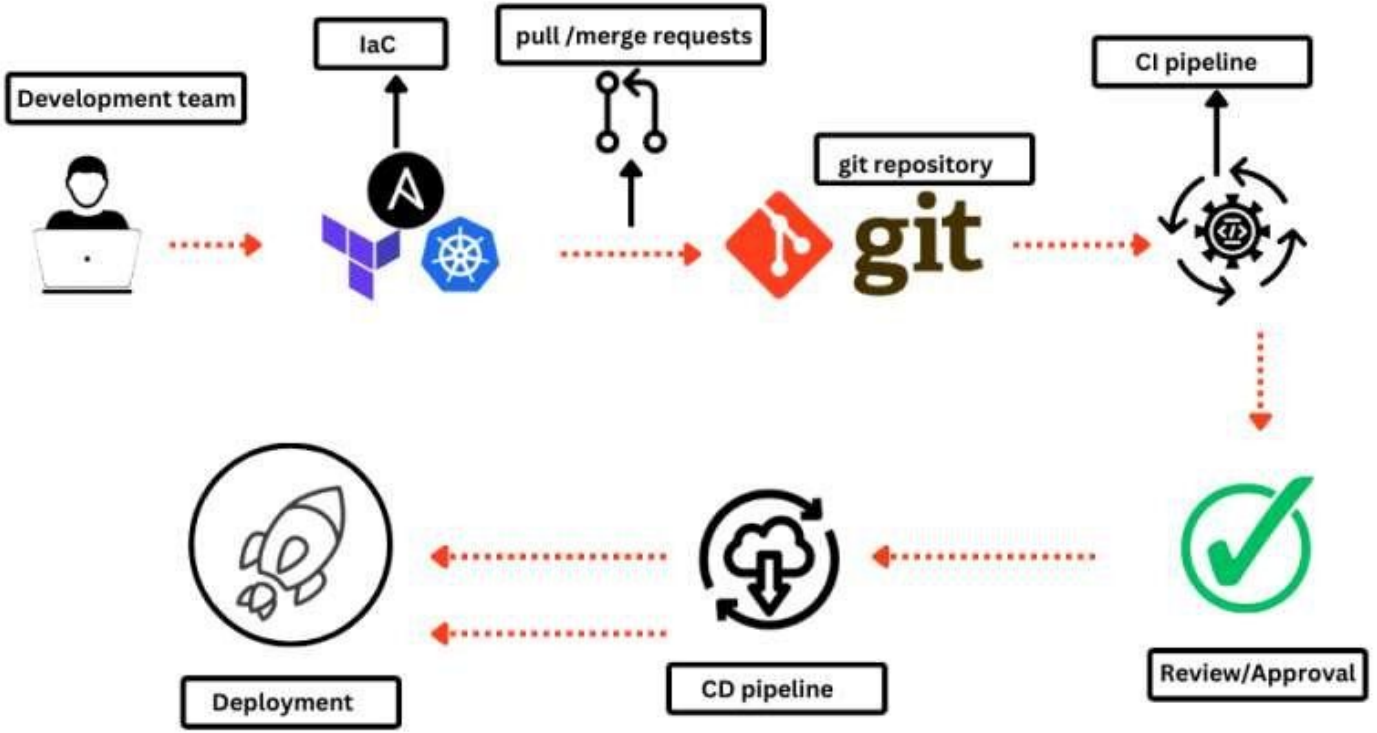
Continuous Delivery

- What is it?
  - The practice of **making software always ready for release to production**, *so deployments can happen on demand*.
- Motivation
  - **Smaller, safer changes**
  - **Faster validation** with users
  - Business can demand **deploy anytime**
  - **Minimize “big bang” releases**
  - Higher confidence in **release readiness**
- The core idea
  - *Always be ready to deploy to production (specially Fridays)*

Continuous Delivery

- What is it?
  - The practice of **making software always ready for release to production**, *so deployments can happen on demand*.
- Motivation
  - **Smaller, safer changes**
  - **Faster validation** with users
  - Business can demand **deploy anytime**
  - **Minimize “big bang” releases**
  - Higher confidence in **release readiness**
- The core idea
  - *Always be ready to deploy to production (specially Fridays)*

GitOps Workflow



Continuous Delivery

- Key dimension: **Delivery strategies**.
- Release ≠ Deployment
  - Code may be deployed continuously, but released in controlled ways.
- Delivery strategies define how and when users see changes.

Types of Delivery

- Big Bang Delivery
- Canary Releases
- Blue-Green Deployment
- Rolling Updates
- Feature Toggles (Flags)
- A/B Testing (Experimentation)
- Dark Launches

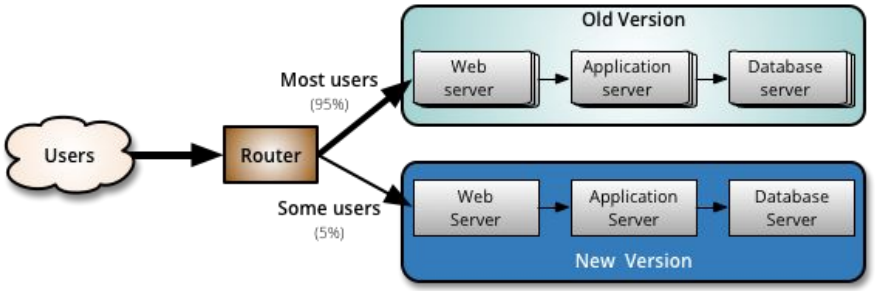
Big Bang Delivery

- All users get the update at once.
- Simple, but high risk.
- Common in legacy systems.



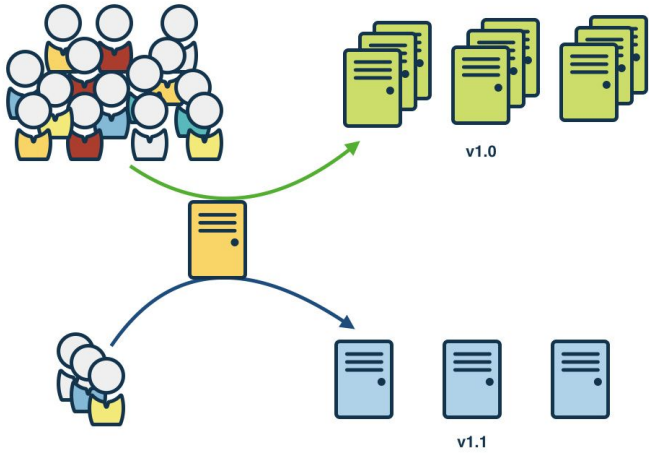
Canary Releases

- Release to a **small subset** of users first.
- **Monitor** metrics (errors, latency, business KPIs).
- Roll back quickly if issues.
- Lowers blast radius.



Blue-Green Deployment

- Two identical environments: Blue (live) and Green (idle).
- Deploy to Green, then switch traffic.
- Enables zero-downtime releases and easy rollback.

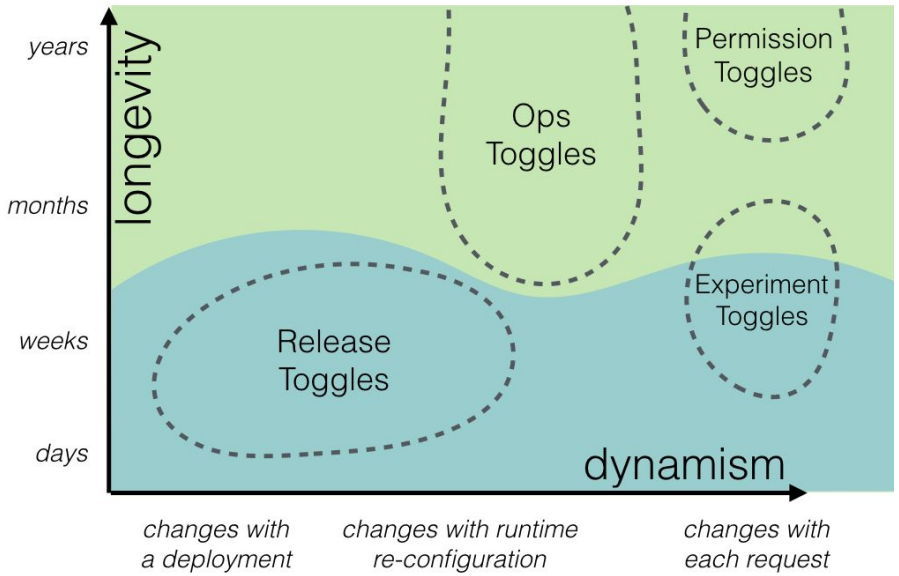


Rolling Updates

- Gradual rollout server by server (or pod by pod).
- No downtime.
- Less drastic than Blue-Green.
- Harder to roll back.

Feature Toggles

- Code deployed, but **features hidden** until activated.
- Enables **trunk-based** development.
- Supports **testing in production**.
- Must manage **toggle debt**.



A/B Testing

- Variant releases to **test user behavior**.
- **Driven by metrics and experimentation** platforms.
- **Combines deployment with product validation**.

Dark Launches

- Deploy feature to production but **invisible to users**.
- **Test performance** under real load.
- **Useful for backend-heavy** features.

Takeaways

- Choose delivery type based on **risk tolerance**, **user base**, and system **architecture**.
- Often, strategies are **combined** (e.g., Canary + Feature Flags).
- Continuous Deployment is about **speed with safety**.

Takeaways

Strategy	Risk	Speed	Rollback	Use Case
Big Bang	High	Fast	Hard	Small apps
Canary	Low	Medium	Easy	Large scale systems
Blue-Green	Low	Fast	Easy	Critical services
Rolling	Medium	Medium	Hard	Cloud-native apps
Feature Toggles	Low	Fast	Easy	Continuous experimentation
A/B Testing	Low	Slow	N/A	Product validation
Dark Launches	Low	Fast	Easy	Performance testing

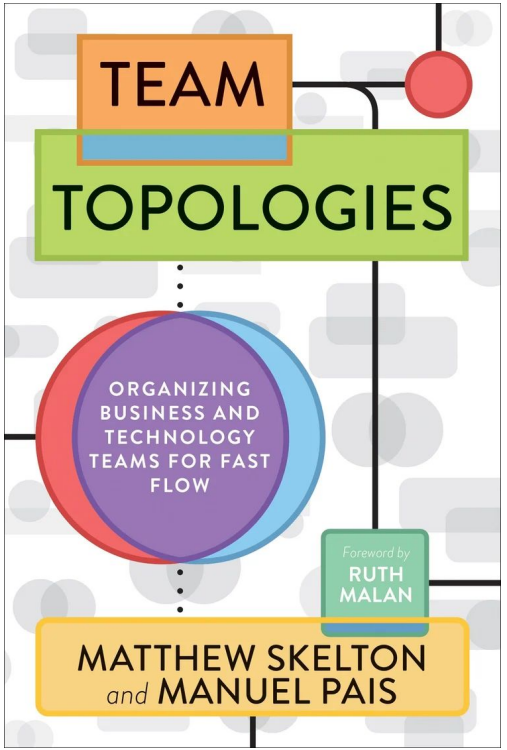
Dudas?

Agenda

1. DevOps
2. **Sidetrack: Team Topologies**
3. Continuous Integration
4. Continuous Delivery

Sidetrack: What is Team Topologies?

- A **framework** for organizing and structuring teams in a way that improves collaboration, autonomy, and software delivery.
- Key Principles
  - Teams are organized to optimize the **flow of work**.
  - **Clear patterns** for team interactions (collaboration, X-as-a-service, and facilitating).
  - **Four fundamental team types** (Enabling, Stream-aligned, Complicated Subsystem, and Platform).
- Benefits
  - Improved team **autonomy**
  - Enhanced delivery **speed**
  - Better **alignment** with business needs



4 fundamental topologies

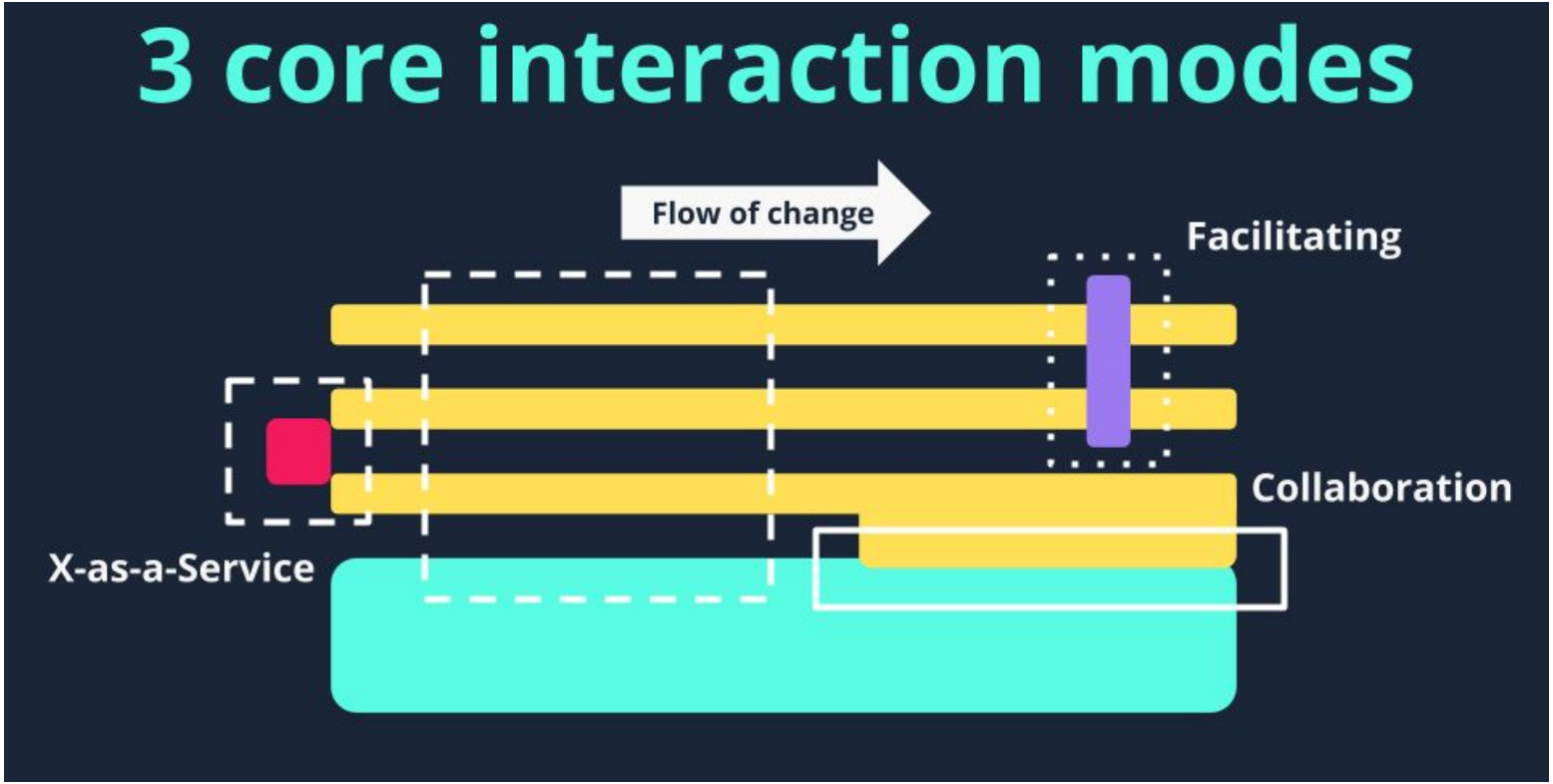


Stream-aligned team

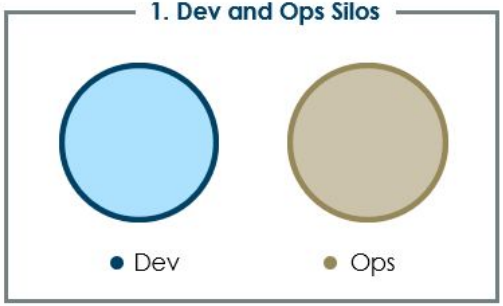
Enabling team

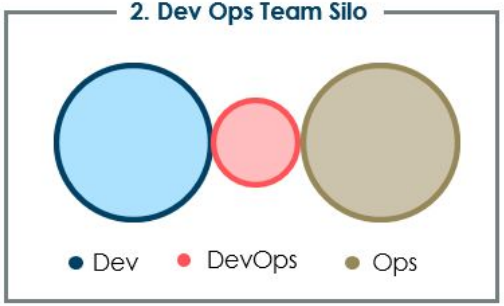
Complicated Subsystem team

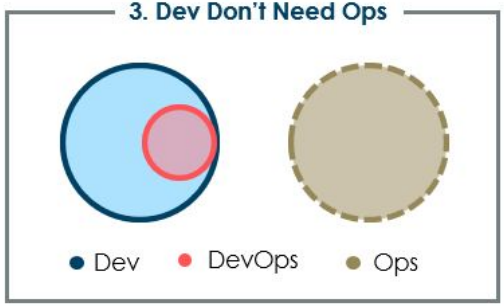
Platform team

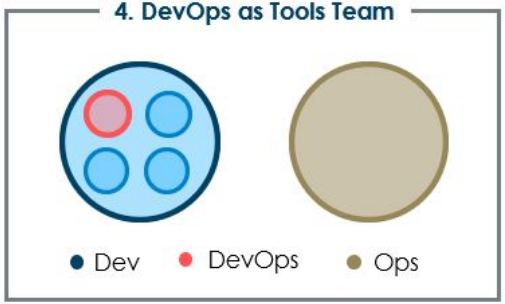

3 core interaction modes
Flow of change
Facilitating
Collaboration
X-as-a-Service

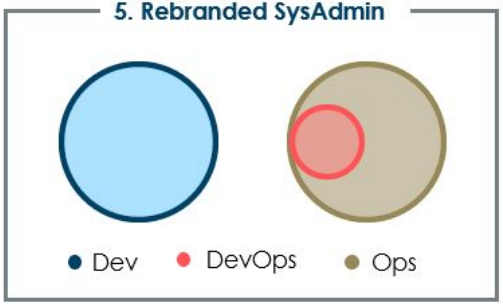
Anti Patterns

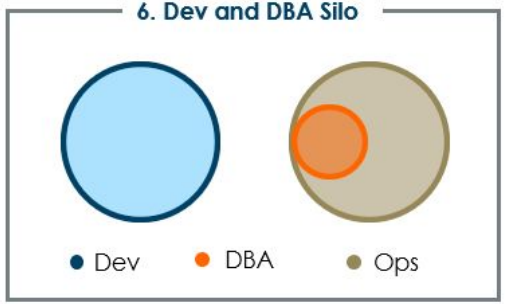












Patterns

Dev & Ops Collaboration

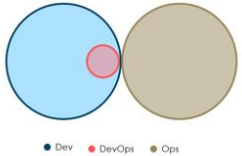
As-Is Model



- Need substantial organization change to establish it and a good degree of technical competency in each product team
- Difficult to scale at enterprise level
- Requires high level of collaboration
- Chances of failures are high if shared goals are not clearly owned by development and operations team
- Team needs to acquire required skills to understand each other's work and seek inputs
- Difficult to achieve standardization of process & tooling, as team do not have an organization level visibility

Embedded Infra DevOps

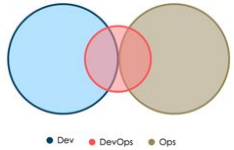
Potential Target



- Platform team delivers an agile DevOps infrastructure (IaaS) for multiple product teams to consume
- Team within dev owns and manages operational features, provisioning, metrics, monitoring
- This team communicates with Platform Ops (IaaS) team
- Less collaboration, easy to implement, quick value
- Requires initial maturity to have a stable platform with a dedicate Platform Ops team
- Difficult to scale and standardize across enterprise if initial maturity is low

DevOps As Shared Service

Potential Target



- Suitable for organizations that tend to have(or tendency) gap between Development & Operations team
- Well-defined ownership to drive agility and DevOps practices and adoption across the organization
- Helps standardize the process, tool chain
- Focus on automation-first DevSecOps framework
- Brings strategy and execution under one umbrella to improve visibility across SDLC
- A great way to manage scale and stability
- Product team not fully autonomous